2022 IEEE 28th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA) | 978-1-6654-5344-8/22/$31.00 ©2022 IEEE | DOI: 10.1109/RTCSA55878.2022.00027

Enabling Real-time AI Inference on Mobile Devices via GPU-CPU Collaborative Execution

Hao Li*, Joseph K. Ng*, Tarek Abdelzaher†
Department of Computer Science, Hong Kong Baptist University, Hong Kong*
Department of Computer Science, University of Illinois at Urbana-Champaign, IL, USA†
Email:{cshaoli, jng}@comp.hkbu.edu.hk*, zaher@illinois.edu†

***Abstract*—AI-powered mobile applications are becoming increasingly popular due to recent advances in machine intelligence. They include, but are not limited to mobile sensing, virtual assistants, and augmented reality. Mobile AI models, especially Deep Neural Networks (DNN), are usually executed locally, as sensory data are collected and generated by end devices. This imposes a heavy computational burden on the resource-constrained mobile phones. There are usually a set of DNN jobs with deadline constraints waiting for execution. Existing AI inference frameworks process incoming DNN jobs in sequential order, which does not optimally support mobile users' real-time interactions with AI services. In this paper, we propose a framework to achieve real-time inference by exploring the heterogeneous mobile SoCs, which contain a CPU and a GPU. Considering characteristics of DNN models, we optimally partition the execution between the mobile GPU and CPU. We present a dynamic programming-based approach to solve the formulated real-time DNN partitioning and scheduling problem. The proposed framework has several desirable properties: 1) computational resources on mobile devices are better utilized; 2) it optimizes inference performance in terms of deadline miss rate; 3) no sacrifices in inference accuracy are made. Evaluation results on an off-the-shelf mobile phone show that our proposed framework can provide better real-time support for AI inference tasks on mobile platforms, compared to several baselines.**

I. Introduction

The resurgence of artificial intelligence in the past decade offers one of the most popular and promising research directions for mobile computing due to its impressive performance advances on many mobile tasks ($e.g.$ computer vision). Compared with traditional sensing and signal processing approaches, AI models often have strong feature extraction and prediction capabilities. This helps to increase automation levels and improve operating efficiency of many applications, such as self-driving vehicles [1] and intelligent manufacturing [2]. Inspired by the increasing success of AI, fields of mobile computing and mobile sensing have recently embraced research solutions that exploit machine intelligence to provide accurate and robust inference results [3], [4], [5], [6].

We are interested in applications where mobile AI inference workloads are executed on end-devices. Computing close to the edge can reduce the communication latency and ensure privacy for mobile sensory data [7]. However, mobile devices usually don't have sufficient computation and memory resources to support heavy AI workloads, especially when running multiple DNN jobs. More importantly, in practice, end users need to interact with mobile applications in a real-time manner. Most recent research published in mainstream AI venues ignores real-time requirements of mobile AI jobs. Take an augmented reality (AR) application as an example [8]. The AR application must recognize the motion of hands, understand what users say, and analyze the physical environment. These tasks map to different DNN jobs, such as hand tracking, speech recognition, and image segmentation. These DNN jobs have real-time requirements. In other words, they must finish within specified deadlines. Otherwise, the perceived quality of service is degraded. For example, users cannot generally tolerate a response time of 2 seconds or more for hand recognition tasks. We instead seek to schedule the multiple DNN jobs to meet deadline constraints. Today's mobile phones are equipped with powerful multi-core CPUs and dedicated accelerators (such as GPU) [9]. Thus, we focus on the exploration of heterogeneous processors on mobile phones for AI inference.

In this paper, we focus on dividing the workload between CPUs and GPUs for improved timeliness and quality. As this work targets the mobile platform, we provide measurement studies on mobile phones to motivate the investigation. Our measurement results reveal that, 1) heading layers in DNN models require higher levels of computational resources when compared to tailing layers; and 2) mobile CPUs can provide competitive AI inference performance compared with mobile GPUs for the smaller and less-paralllizable trailing layers. Based on these observations, we propose a framework to promote GPU/CPU collaboration for mobile AI inference with real-time requirements. A two-stage execution setting is adopted in our system, where a mobile GPU executes the first part of a DNN job, which may consume more computational resources, and the mobile CPU executes the remaining workload of the DNN job. Thus mobile GPU is better utilized whereas the mobile CPU can participate in the DNN inference process at later stages that are less parallel.

Assuming that each DNN job has a deadline when it arrives, the objective of our real-time scheduler is to make as many jobs as possible finish before their deadlines in the scheduling period. Using an Earliest Deadline First (EDF) policy, we formulate a utility maximization problem and transform it into an Integer Linear Programming (ILP) form. We then design a dynamic programming (DP) based approach to obtain the optimal partitioning solution for DNN jobs. Furthermore, we propose other algorithms as baselines of the DP-based

2325-1301/22/$31.00 ©2022 IEEE
DOI 10.1109/RTCSA55878.2022.00027

Authorized licensed use limited to: SHERIDAN LIBRARY. Downloaded on September 17,2026 at 09:37:32 UTC from IEEE Xplore. Restrictions apply.

algorithm for comparison purposes. We implement the proposed framework in an off-the-shelf mobile phone. A series of experiments are conducted to demonstrate the effectiveness of the proposed framework and the DP-based algorithm.

We summarize our main contributions of this work as follows.

- We study multiple DNN jobs' execution with real-time requirements on the mobile platform.
- We propose a new framework for DNN inference. The proposed framework enables a mobile GPU/CPU collaboration by partitioning the DNN workloads among the CPU and GPU in a manner that better utilizes the computation resources on the mobile phone.
- We formulate the real-time DNN scheduling problem and transform it into an integer linear programming problem. We design a dynamic programming based algorithm that can solve our problem by computing an optimal partitioning.
- We implement the system by extending TensorFlow Lite. Several state-of-the-art DNN models are applied in our experiments. We find that our framework and algorithm achieve good performance and provide real-time support for mobile AI services.

The rest of this paper is organized as follows. In Section II, background and motivation are introduced. The design of the proposed system is given in Section III. Section IV and V present the problem formulation and detail a dynamic programming solution approach. The implementation and evaluation are presented in Section VI. Section VII overviews related work. Finally, we conclude in Section VIII.

II. Background & Motivation

A. Deep neural networks

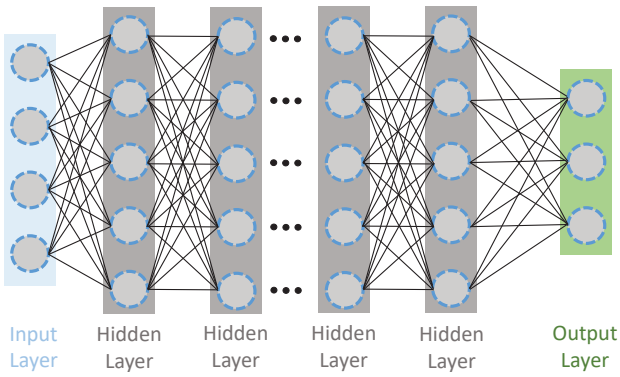


Fig. 1. An example of DNN with multi-layer structure

Deep neural networks are representation learning models with multi-layer structures [10]. Neural neurons are basic computation units and compose layers of DNN. As Fig. 1 shows, neurons in each layer learn representations and capture features from previous layers, and computed inter results are transferred to subsequent layers. The execution process of DNN is layer-wise forward propagation, from inputs to outputs

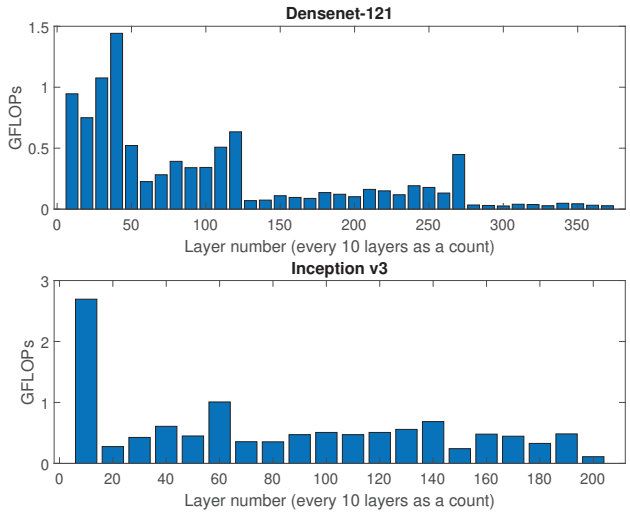


Fig. 2. FLOPs of layers in two DNN models

through hidden layers. Also, the layered sequence, the depth and the number of neurons are fixed and preordained by design before training the model. Thus, DNN can be viewed as a directed graph with vertices (organized neural layers) and edges (inter-data dependencies) [11]. This understanding provides us an opportunity to partition the DNN into several parts to achieve specific optimization goals.

Modern neural networks usually contain millions of parameters ($e.g.$, weights and biases). When these networks are deployed in mobile devices, a lot of computational resources and memory space are required to conduct multiple-level abstraction from complex data [12]. Starting from the input layer, a DNN transforms high-dimensional data, like images, into representations in a more and more high-level and abstract manner. The output layer finally computes the inference result based on the relatively low-dimensional features from hidden layers. Fig. 2 shows FLOPs of layers with some typical DNN models. We can see that most heading layers have a larger amount of FLOPs while tailing layers have fewer. Note that different models have their unique designs and may not follow this pattern. However, in general, the initial layers will consume more resources compared with the deeper layers. This motivates the mapping of layers into suitable processing units in a cost-efficient way.

B. Mobile hardware

Mobile devices have experienced rapid development in their GPU acceleration abilities. However, a large performance gap between today's desktop GPUs and mobile GPUs still exists. One of the main reasons for this gap is the power consumption difference (up to 50x) [13]. A desktop GPU usually contains thousands of cores and has higher clock speeds. In contrast, a mobile GPU only has tens of cores or even less. The clock speed is restricted due to stringent thermal constraints and battery life. To date, most mobile devices are equipped with multi-core CPUs for general-purpose computing. Although

Authorized licensed use limited to: SHERIDAN LIBRARY. Downloaded on September 17,2026 at 09:37:32 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DNN models' execution latency on mobile GPU and CPU (Samsung A71)

DNN \ processor	mobile GPU	mobile CPU (4 threads)
Inceptin v3	$125ms$	$274ms$
Densenet-121	80 ms	$177ms$
MobileNet v1	$16ms$	$29ms$
Resnet-50	$81ms$	$172ms$

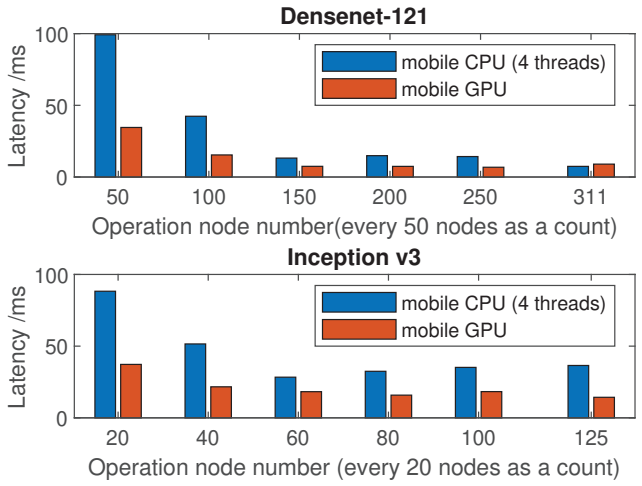


Fig. 3. DNN models' layer-wise execution latency on mobile GPU and CPU (Samsung A71)

the CPU is not as good at matrix multiplication (a core computation during AI inference), it can still play a role when compared to mobile GPUs. Table. I shows the execution time of different DNN models on mobile GPUs versus CPUs. The layer-wise latency is also illustrated in Fig. 3. Mobile CPUs provide comparable performance, which motivates investigating GPU-CPU collaborative execution to boost the inference performance, instead of only using the GPU.

Apart from the performance gap, today's mobile GPUs get limited support from computing architecture and inference frameworks when compared with desktop GPUs. This prevents some state-of-the-art techniques [14] [15] from being exploited on mobile platforms. For example, the CUDA Multi-process system supports spatial sharing on (desktop) GPUs, which allows kernels from different DNNs to run in parallel [16]. This multiplexing allows unused GPU resources to be utilized by other DNNs, thereby improving overall utilization on desktop GPUs. Unfortunately, spatial sharing is not broadly supported on mobile GPUs at this time. Furthermore, temporal sharing of mobile GPUs is not directly supported in existing mobile deep learning inference frameworks. Let's take TensorFlow Lite (TFLite) (designated for mobile) as an instance. Fig 4 shows how multiple DNN jobs work in OpenCL-based TFLite with mobile GPU. After pre-processing on a CPU, operations in DNN ($i.e.$, CONV_2D, Fully_Connected), also called nodes in TFLite, will be executed on the GPU sequentially and then transfer the result to the CPU for further post-processing. Note that, some operations are not supported by TFLite with GPU delegate, especially custom operators. In this circumstance (like DNN2 in Fig. 4), TFLite moves unsupported operations and the remaining nodes to the CPU for further execution. Except for this involuntary case, one DNN job has to wait until another DNN job yields control on GPU. This may result in under-utilization of mobile GPUs when some nodes in the DNN require few computational resources and have a low degree of parallelism. Additionally, the serialized processing is limited in its consideration for executing multiple DNNs with timing constraints, such as inference in Fig. 4, where only DDN1 could meet its deadline. Furthermore, idle CPU resources cannot be well utilized for DNN inference. A feasible solution is to offload some DNN workloads to the CPU. As Fig. 5 illustrates, if DNN1 offloads some workload to the CPU, then DNN2 and DNN3 could occupy the GPU earlier and the latency requirements of the three jobs are all satisfied.

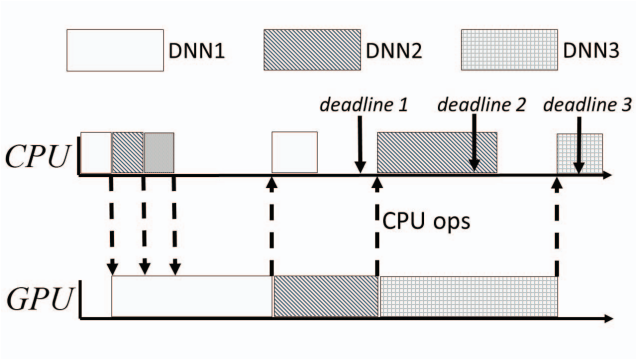


Fig. 4. Multiple DNN inference in TFLite

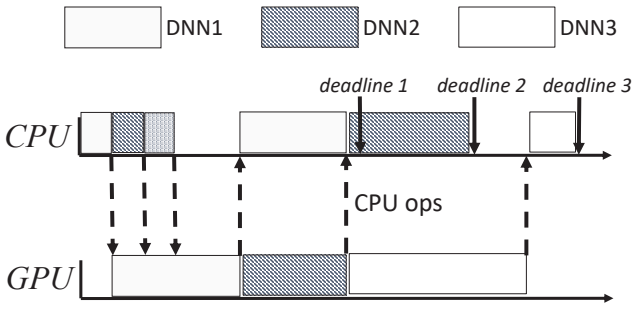


Fig. 5. Multiple DNN inference with CPU offloading

III. System Design

In this section, we introduce how our system is designed to better utilize computational resources and provide real-time support for AI tasks.

Authorized licensed use limited to: SHERIDAN LIBRARY. Downloaded on September 17,2026 at 09:37:32 UTC from IEEE Xplore. Restrictions apply.

A. Two-stage execution

As described in Sec. II, mobile GPU architecture and existing mobile inference frameworks lack real-time support for multiple DNNs' inference on heterogeneous resources. If we assume a DNN can be arbitrarily partitioned into two stages, then it is intuitive to propose a two-stage execution setting for mobile GPU/CPU collaboration (such as DNN1 in Fig. 5). A mobile GPU is an accelerator for DNN inference. It can undertake the first part of a DNN job, which may consume more computational resources and is generally more parallel. The second stage is executed by the mobile CPU. Within the two-stage execution setting, first, the mobile GPU is efficiently utilized because it handles the part of the workload that has higher resource requirements. Second, concurrent execution of GPU and CPU is enabled when processing different DNN jobs. Third, the contention of mobile GPUs can be dynamically controlled by DNN partitioning and tail offloading (to the CPU), which is actually a time-sharing operating on mobile GPUs. This can facilitate earlier GPU occupation and completion of DNN jobs in time. Note that, in our work, we assume that part of CPU cores in ARM big.LITTLE architecture are idle (such as one cluster), which can be utilized for DNN inference without causing resource contention with the mobile operating system and other applications in the mobile phone. We regard these idle cores as a processing unit and use them for second stage execution.

B. System architecture

Fig. 6 presents an overview of our system architecture. Let there be m DNN jobs at time t in the mobile phone with real-time constraints. Assume that the layer-wise execution latency profiling for different DNN models (on both GPU and CPU) is completed in advance. Additionally, the computational resources' available time is given. The scheduler integrates all information and requirements together, then makes the scheduling decision to enable mobile GPU/CPU two-stage collaboration execution. First, the scheduling decision determines the priority of DNN jobs. In this work, we take Earliest Deadline First (EDF) as the basic scheduling policy. The priority of each DNN job is assigned inversely proportional to its absolute deadline, and all DNN jobs are sorted into a queue awaiting execution. Next, with the execution order fixed, the scheduler partitions DNN jobs and offloads workloads to CPU based on their execution latency profiling and real-time requirements. Last, the scheduler is invoked at the beginning of scheduling period H. When a new set of DNN jobs arrive at time $(t + H)$, the scheduler checks whether any job is still awaiting execution in the queue, and then gathers them with new arrivals into a new set of jobs for the next scheduling decision making.

C. Utility maximization

As previously defined, in a scheduling period H, there is a set of m sporadic DNN jobs associated with deadlines. Abandoning a job is not allowed in this system. Following EDF policy, the goal of the scheduler is to complete more

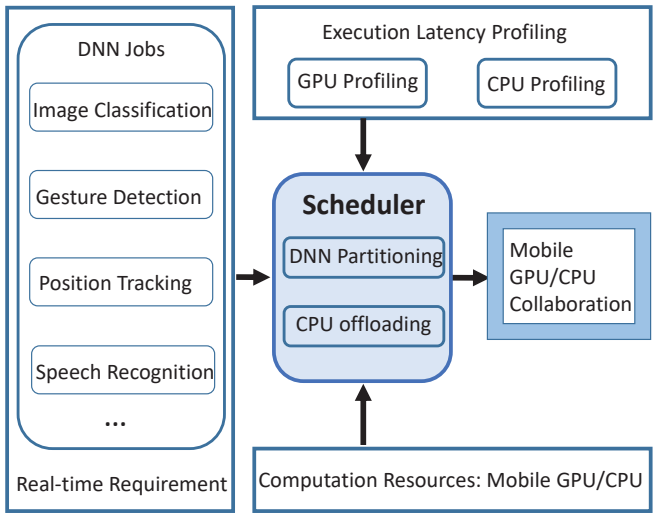


Fig. 6. System architecture

DNN jobs by their respective deadlines, which provides a better user experience. If a DNN job can meet its deadline, we set the utility to be one. Otherwise, the utility is zero due to degradation in the quality of service. The scheduler aims to achieve a maximum of utility $\sum_{j=1}^{m} R_j$ for the m DNN jobs in a scheduling period under EDF. The design space is the partitioning point for each DNN job. We formulate this utility maximization problem and propose algorithms to solve it in following sections.

D. Execution latency profiling

The execution latency profiling of DNN models can be performed either online or offline. Several research efforts explore online profiling for layer-wise DNN execution latency. Online profiling builds a regression model to estimate the latency based on operation types, input data size, number of parameters, processing cores and other factors. However, they have high prediction errors as reported on real mobile devices [12][17]. In contrast, offline profiling is more accurate as it can be based on a longer training stage. The reason for this is that spatial-sharing of GPU is unsupported in mobile devices and we also assume the chosen idle CPU cores are dedicated for DNN jobs. Resource contention would not happen in DNN execution period. Thus, offline profiling can provide stable and accurate estimations of latency.

However, it is time-consuming to profile layer-wise execution time with different configurations, as DNN models usually contain hundreds of layers (nodes). To facilitate this process, we add some potential partitioning points in DNN models as Fig. 7 illustrates. There are a series of nodes between two partition points, and we call this a block. This block-wise profiling has is coarser-grained compared to layer-wise profiling. To prevent side effects on scheduling, we make the blocks small compared to the total execution time.

Authorized licensed use limited to: SHERIDAN LIBRARY. Downloaded on September 17,2026 at 09:37:32 UTC from IEEE Xplore. Restrictions apply.

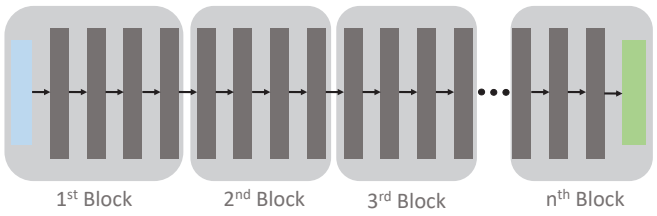


Fig. 7. The illustration of DNN Partitioning

IV. Problem formulation

In this section, we formulate the utility maximization problem studied in the paper. Let's sort m DNN jobs in EDF manner, and denote the job queue as J, where $j \in \{1, 2, ...m\}$ is the index with the deadline in ascending order. The scheduler decides the partitioning solution $\mathcal{I}$ of J, where $\mathcal{I}_j^k$ is a 0-1 variable, indicating whether to terminate the computation of the $j\text{-}th$ job on GPU at partition point k. We have $\sum_{k=1}^{N_j} \mathcal{I}_j^k = 1$ because each job only has one partition point in the two-stage execution. We assume that input data is pre-processed, such as resizing, and ready for first stage execution. Thus, the DNN job starts with GPU at time $\hat{g_j}$ and finishes its first stage in GPU at $\ddot{g_j}$. Next, it executes its second stage on CPU from $\hat{c_j}$ to $\ddot{c_j}$, including the remaining workloads and post-processing, such as converting the format of inference output. Below is the formulation of the optimization problem.

$$\max_{\mathcal{I}} \sum_{j=1}^{m} R_j$$

$$s.t. \qquad \mathcal{I}_j^k \in \{0,1\}, \sum_{k=1}^{N_j} \mathcal{I}_j^k = 1, \forall j \tag{1}$$

$$\hat{c_j} \geq \ddot{g_j}, \forall j \tag{2}$$

$$\hat{g_j} \geq \ddot{g_{j-1}}, \hat{c_j} \geq \ddot{c_{j-1}}, \forall j > 1 \tag{3}$$

$$\ddot{g_j} = \hat{g_j} + G_j \cdot \mathcal{I}_j, \ddot{c_j} = \hat{c_j} + C_j \cdot \mathcal{I}_j, \forall j \tag{4}$$

$$R_j = \begin{cases} 1 & \ddot{c_j} \leq d_j, \\ 0 & \ddot{c_j} > d_j, \end{cases} \forall j \tag{5}$$

where (2) indicates that the second stage of current job must be executed after completion of the first stage; (3) indicates that execution of DNN jobs must satisfy the job precedence constraints both on GPU and CPU; (4) indicates how execution time is computed from DNN partitioning solution $\mathcal{I}_j$ and GPU/CPU latency profiling (G_j, C_j); (5) indicates the utility function of the problem as described in Sec. III-C. Also, we can convert (5) into linear constraints by adding a large-enough positive constant $\mathcal{M}$, as shown below.

$$\begin{aligned} \mathcal{M} R_j &> d_j - \ddot{c_j}, \\ \mathcal{M}(1 - R_j) &\geq \ddot{c_j} - d_j, \\ R_j &\in \{0,1\} \end{aligned} \tag{6}$$

Thus, the scheduling problem is transformed into an Integer Linear Programming (ILP) problem. Next, a dynamic programming solution approach is presented to find the optimal partitioning points in polynomial time.

TABLE II
Notations adopted in this paper

Notations	Description
j	Index of the job
J	Job queue
H	Scheduling period
m	Total job amount in H
ξ	Basic time unit
N_j	Number of blocks for $j\text{-}th$ job
d_j	Deadline of $j\text{-}th$ job
D	Deadline set
$\mathcal{I}$	DNN partitioning solution for J
$\mathcal{I}_j$	DNN partitioning solution of $j\text{-}th$ job
$\mathcal{I}_j^k$	Whether offload at $k\text{-}th$ partition point of $j\text{-}th$ job
$\hat{g_j}$	Start time of $j\text{-}th$ job on GPU
$\ddot{g_j}$	Finish time of $j\text{-}th$ job on GPU
$\hat{c_j}$	Start time of $j\text{-}th$ job on CPU
$\ddot{c_j}$	Finish time of $j\text{-}th$ job on CPU
G_j	GPU latency profiling of $j\text{-}th$ job
C_j	CPU latency profiling of $j\text{-}th$ job
g_j^k	The $j\text{-}th$ job execution time on GPU from $1\text{-}st$ block to $k\text{-}th$ block
c_j^k	The $j\text{-}th$ job execution time on CPU from $(k+1)\text{-}th$ to $N_j\text{-}th$ block
R_j	Utility of $j\text{-}th$ job
S_j	State set of the top j job
S	All state sets

V. A dynamic programming approach

Dynamic programming is an approach that converts the complex problem into a sequence of small problems, and obtains the solution by solving each of those small problems using recurrence relations. Below, we describe a dynamic programming based algorithm to obtain the optimal solution of the optimization problem.

A. System state

In this system, time is defined as the multiples of some basic time unit ξ, thereby all time parameters in this paper are represented in integer format. For example, time in a scheduling period H is represented as $\{0, 1, 2, ...H\}$, which are $\{0, \xi, 2\xi, ...H\xi\}$ actually. Let x, y denote GPU time and CPU time, respectively. The addition or subtraction on x, y

Authorized licensed use limited to: SHERIDAN LIBRARY. Downloaded on September 17,2026 at 09:37:32 UTC from IEEE Xplore. Restrictions apply.

can express any time instant on GPU/CPU. The discretization on time can help us define our problem as a dynamic program.

As mentioned, DNN jobs are sorted in the EDF manner. Higher index jobs will be executed after lower indexes jobs. Also, no DNN jobs are dropped. Therefore, we cast the problem into a dynamic program by dividing it into m stages. In $j\text{-}th$ stage, we focus on the execution of top j DNN jobs from the job queue J, which requires a partitioning decision. This decision partitions each of the top j jobs into GPU and CPU.

Each stage is associated with states. We define the stage index j, together with GPU and CPU time (x, y) as the state of the system. The state (j, x, y) uniquely identifies a certain position of the whole problem, and we need to calculate the optimal decision for it. In this problem, (j, x, y) represents the subproblem that needs to finish the top j DNN jobs in order, while the execution doesn't exceed the time limit x and y on GPU/CPU. There could be multiple partitioning solutions making the top j jobs finished within (x, y). We choose the partitioning solution to achieve the maximum rewards as the optimal decision, that is to say, the partitioning solution makes the maximum number of jobs meet the deadlines. We denote the optimal decision as $I(j, x, y)$ and the corresponding maximum reward as $R(j, x, y)$. On the other hand, if there is no partitioning solution completing the top j jobs within the given time (x, y), then we set $I(j, x, y)$ to be $\varnothing$ and $R(j, x, y)$ to be $-\infty$. Originally, the $j\text{-}th$ stage is associated with a number of states from $(j, 0, 0)$ to (j, H, H). However, the top j jobs have execution time upper bounds (UG_j,UC_j) on GPU and CPU. In other words, to complete the top j jobs, we occupy GPU and CPU UG_j and UC_j units of time at most. UG_j and UC_j are simply obtained by allocating all possible workloads of the top j jobs to GPU or CPU. Also, note that $y \geq x$ always holds as the post-processing workloads must be done in CPU. Therefore, for the $j\text{-}th$ stage, we only need to consider the state (j, x, y) ranging from $(j, 0, 0)$ to (j, UG_j, UC_j), where $y \geq x$. We use set S_j to denote these states in the $j\text{-}th$ stage, and let $S = \{S_1, S_2, ...S_m\}$.

B. State transition equations

Assume that we have solved all the subproblems in the $(j-1)\text{-}th$ stage and obtained the optimal reward and decision for each state in S_{j-1}. Then for the next stage, the $j\text{-}th$ stage, we can calculate $R(j, x, y)$ and $I(j, x, y)$ for states in S_j by the state transition between $j\text{-}th$ and $(j-1)\text{-}th$ stage. Suppose we are in the state (j, x, y) and trying to find the optimal partitioning solution for the top j job under time limit (x, y). For the j-th job, there are N_j partitioning points, and the $k\text{-}th$ partitioning point makes the $j\text{-}th$ job occupy g_j^k time on GPU and c_j^k time on CPU separately. We denote $\acute{x}_k$ and $\acute{y}_k$ as the end time of top $(j-1)$ jobs on GPU and CPU when the $k\text{-}th$ partitioning point is selected for the $j\text{-}th$ job. We make y as the end time of the $j\text{-}th$ job, and then analyze the state transitions in two different cases.

- Case 1: $x \leq y - c_j^k$. In this case, we can get $\acute{x}_k$ and $\acute{y}_k$ by substracting g_j^k, c_j^k from x, y directly.
- Case 2: $x > y - c_j^k$. For the j-th job, CPU must wait for the finish of GPU part and then can continue executing the remaining workloads. Therefore, when $x > y - c_j^k$, the execution of the j-th job should start at $y - c_j^k - g_j^k$ on GPU. Otherwise, the execution will exceed the time limit (x, y).

We can summarize two cases into equations as the following.

$$\acute{x}_k = \min(x, y - c_j^k) - g_j^k, \quad \acute{y}_k = y - c_j^k$$

By applying the partitioning point k, the state is transited to $(j-1, \acute{x}_k, \acute{y}_k)$. If $R(j-1, \acute{x}_k, \acute{y}_k)$ is greater than $-\infty$, which means the top $j-1$ job can be finished within $(\acute{x}_k, \acute{y}_k)$, and thus the top j jobs can be finished under time limit (x, y). Then the corresponding k can be a candidate partitioning point for the $j\text{-}th$ job. Since y is the end time of the $j\text{-}th$ job, $\mathcal{R}_j$ is fixed in the state (j, x, y) regardless of which candidate partitioning point is chosen. Therefore, the maximum reward $R(j, x, y)$ can be obtained from the maximum $R(j-1, \acute{x}_k, \acute{y}_k)$, where $k \in \{1, ..., N_j\}$. We denote the optimal partition point for the $j\text{-}th$ job in state (j, x, y) is k_o, and the optimal decision can be achieved by adding k_0 into $I(j-1, \acute{x}_{k_o}, \acute{y}_{k_o})$. The state transition equation is listed below.

$$k_o = \begin{cases} \underset{k \in \{1,...,N_j\}}{\arg\max}\ R(j-1, \acute{x}_k, \acute{y}_k), \text{if } R(j-1, \acute{x}_k, \acute{y}_k) > -\infty \\ -1, \qquad \text{otherwise} \end{cases}$$

$$R(j, x, y) = \begin{cases} R(j-1, \acute{x}_{k_o}, \acute{y}_{k_o}) + 1, & \text{if } k_o \neq -1 \ \&\ y \leq d_j \\ R(j-1, \acute{x}_{k_o}, \acute{y}_{k_o}), & \text{if } k_o \neq -1 \ \&\ y > d_j \\ -\infty, & \text{if } k_o = -1 \end{cases}$$

$$I(j, x, y) = \begin{cases} \varnothing & \text{if } k_o = -1 \\ I(j-1, \acute{x}_{k_o}, \acute{y}_{k_o}) \leftarrow k_o, & \text{otherwise} \end{cases}$$

C. Algorithm design

We can see the problem satisfies Bellman's principle of optimality [18] as the subsolutions of the final solution are themselves optimal solutions for their subproblems. Therefore, we can employ the dynamic programming based algorithm. The pseudo-code of the proposed dynamic programming based algorithm is shown in Algorithm. 1. It is obvious that states in the first stage are base cases. The base case is simple to solve by enumerating all partitioning points of the $1st$ job. With the subsolutions of base cases, we can handle the subsequent stages by applying the state transition equations and memoizations iteratively. The optimal subsolution in the last stage yields an optimal solution for the entire problem. It is because the subproblem in the last stage is the original problem itself, namely, finishing the top m jobs while achieving the maximum reward. There could be multiple states in the last stage with the same maximum reward. We select the state with the minimum y, and the corresponding partitioning decision as the optimal solution $\mathcal{I}_{opt}$ for our problem.

Authorized licensed use limited to: SHERIDAN LIBRARY. Downloaded on September 17,2026 at 09:37:32 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: The dynamic programming based Algorithm

Input: Job queue J, state sets S, deadline set D
Output: Optimal partitioning solution

if $j = 1$ **then**
  for $(j, x, y) \in S_j$ **do**
    if $\exists k \in \{1, ..., N_j\} \rightarrow (x > g_j^k) \& (y > g_j^k + c_j^k)$ **then**
      $I(j, x, y) = \{k\}$
      update $R(j, x, y)$ based on whether $y \leq d_1$
    else
      $R(j, x, y) = -\infty, I(j, x, y) = \varnothing$
    end
  end
end
for $j = 2, 3, ..., m$ **do**
  for $\{x, y\} \in S_j$ **do**
    update $R(j, x, y)$, $I(j, x, y)$ with state transition equations
  end
end
for *states in the last stage* **do**
  choose the state with maximum reward
  if *exist multiple states* **then**
    choose the state with smallest y
  end
  return the selected optimal solution
end

Algorithm 2: The greedy algorithm

Input: Job queue J, deadline set D
Output: Greedy partitioning solution I

for $j = 1, 2, ..., m$ **do**
  if $\exists k \in \{1, .., N_j\} \rightarrow (\max(c_{j-1}^{\cdot\cdot}, \ddot{g_j}) + c_j^k) < d_j$ **then**
    set $K \leftarrow$all k satisfying the condition above;
    $I \leftarrow \arg\min_{k \in K}(g_j^k)$
  else
    $I \leftarrow \arg\min_{k \in \{1, ..., N_j\}} (\max(c_{j-1}^{\cdot\cdot}, \ddot{g_j}) + c_j^k)$
  end
  update $\ddot{g_j}$, $\ddot{c_j}$
end
return I

VI. Implementation and Evaluation

A. Implementation

We implement the system based on OpenCL-based TensorFlow Lite, which is one of the most widely used mobile deep learning inference frameworks. As described before, existing APIs in TFLite cannot support temporal mobile GPU sharing for a single DNN job. To address this issue, we implement DNN partitioning and CPU offloading. Originally, TFLite has a class $Subgraph$ to define the execution plan for the DNN job, including the type of nodes, execution order and delegate information. When GPU is activated for DNN execution, $GetOpsToReplace()$ is invoked to find which node can be operated on GPU, since some nodes may be unsupported on GPU. Then $GetOpsToReplace()$ returns the index array of eligible nodes, and TFLite uses $ReplaceNodeSubsetsWithDelegateKernels()$ to replace these nodes with delegates and update $Subgraph$. We modify $GetOpsToReplace()$ to return a revised index array, which keeps part of nodes to be executed on the GPU and the remaining ones on the CPU with a specified partitioning point. This extension of TFLite allows us to partition the DNN job and enables mobile GPU/CPU collaborative execution. Other OpenCL-based mobile deep learning frameworks, such as PaddleLite and MACE, have similar workflows to use GPU delegate as an accelerator.

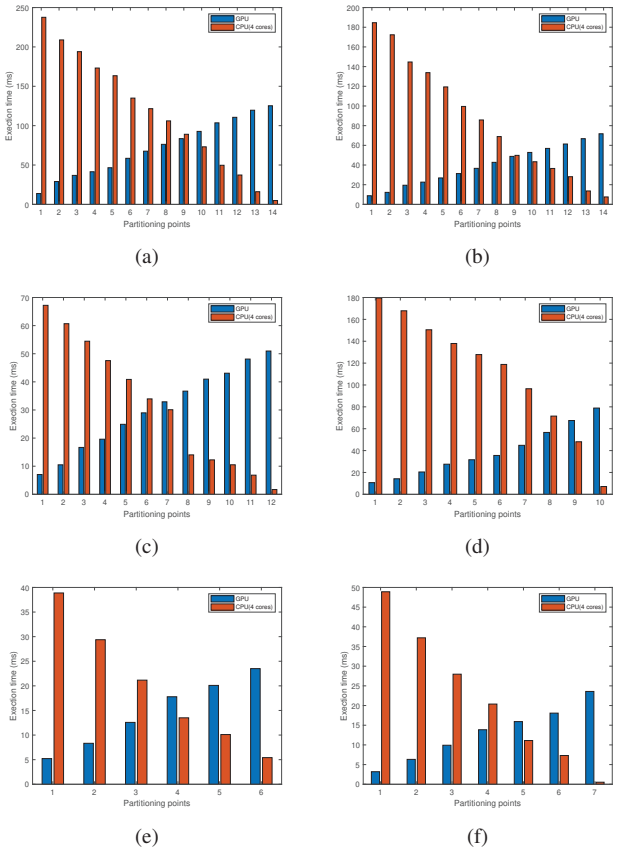


Fig. 8. Latency profiling results for different DNN models ((a) Inception (b) DenseNet-121 (c) FaceNet (d) Resnet-50 (e) Posenet (f) Deeplab)

To evaluate the proposed real-time scheduling framework on a Samsung Galaxy A71 (Snapdragon 730 Soc, 8 CPU cores, Adreno 618 GPU), we choose multiple popular DNN models to imitate the real-world scenarios, including FaceNet [19], Resnet-50 [20], DenseNet-121 [21], Inception v3 [22], PoseNet [23] and Deeplab [24].

Authorized licensed use limited to: SHERIDAN LIBRARY. Downloaded on September 17,2026 at 09:37:32 UTC from IEEE Xplore. Restrictions apply.

B. Baselines

We have the following algorithms as baselines for the dynamic programming based approach.

- **All-GPU**: DNN jobs follow the EDF policy and all workloads are allocated to mobile GPU.
- **Balanced-Partition**: In this algorithm, DNN jobs are partitioned into GPU and CPU in a balanced way. Following the EDF order, we make GPU execution time and CPU execution time of each job to be almost equivalent, so that GPU and CPU can be well pipelined for the collaborative execution.
- **Greedy**: Starting from the DNN job with the earliest deadline, if the job can meet its deadline with some partitioning points, then we choose the point which consumes the fewest GPU resources from these candidates. Hence subsequent jobs can occupy GPU earlier. If a job misses the deadline in any case, we choose the partitioning point whose finish time is the closest to the current deadline, thereby the future jobs may not be delayed too much on the CPU. Following jobs from J will obey this heuristic rule until all jobs are completed. The pseudo-code is given in Algorithm. 2.

C. Evaluation metrics

The following evaluation metrics are used to compare our proposed dynamic programming based approach and other baselines.

1) Deadline miss rate: Although many mobile applications can tolerate a certain amount of deadline misses, it's still significant to provide acceptable performance in terms of deadline miss rate. This ensures a friendly user experience and normal functionality can be maintained. In addition, deadline miss rate is the optimization objective of the proposed dynamic programming based approach, and is the basis of strategies in the greedy algorithm. Therefore, the deadline miss rate is the most important performance metric in the evaluation.

2) Completion rate: As abandoning jobs is disallowed in the system, we need to execute DNN jobs one by one without dropping any of them. When there are a lot of DNN jobs in a scheduling period, it is impossible to finish all of them with limited resources on the mobile phone. Hence, we define completion rate as the percentage of jobs completed within the scheduling period to total jobs. Completion rate is a useful performance metric in this problem, as it evaluates the effectiveness of different schedulers.

D. Performance results

In this subsection, we verify the effectiveness of our proposed approach from evaluation metrics with different experimental settings. Fig. 8 shows how DNN models are partitioned and corresponding execution time on mobile GPU and CPU. Note that, part of CPU cores are used as the CPU resource (one cluster). For each DNN model, the block is set to reduce side effects of scheduling. This offline latency profiling is used as the basis of all experiments. For a scheduling period H (1 second or 2 second), we choose DNNs from the model pool and set deadlines for them from 100ms to the end of H randomly.

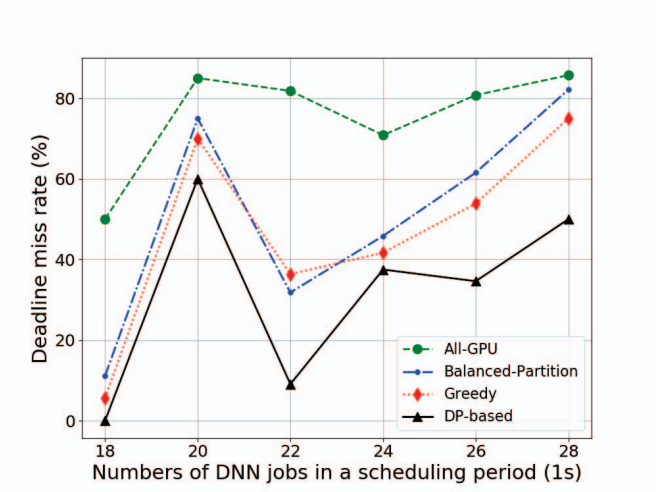


Fig. 9. The deadline miss rate under different numbers of tasks in a scheduling period (1s)

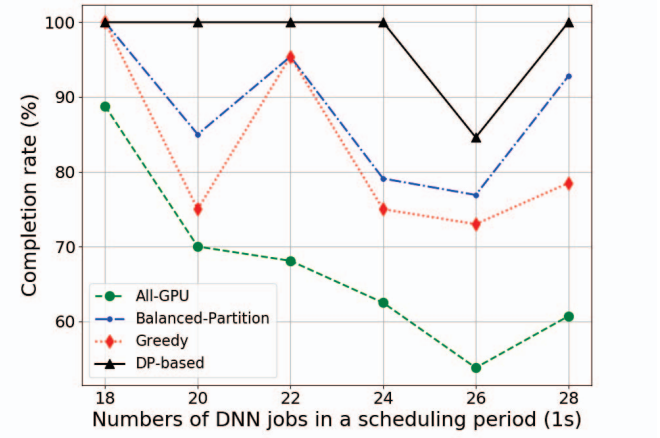


Fig. 10. The completion rate under different numbers of tasks in a scheduling period (1s)

In the first set experiments, we set different numbers of DNN jobs from 18 to 28 with a step of 2 in a scheduling period of 1 second. Fig. 9 shows the deadline miss rate of different algorithms. The DP-based algorithm offers good performance regardless of intensive workloads, $e.g.$, 28 DNN jobs in a scheduling period, while baseline algorithms miss many deadlines. In baseline algorithms, when the workload is light, few deadlines are missed. However, more jobs in a scheduling period cause the performance to drop significantly. Completion rates of different algorithms are shown in Fig. 10. The DP-based algorithm can complete almost all jobs in the scheduling period, while baselines leave many unfinished jobs when workload are heavy. As we choose DNN jobs and their deadlines randomly, the workload intensity of different job sets may not be directly related with the number of jobs, such

Authorized licensed use limited to: SHERIDAN LIBRARY. Downloaded on September 17,2026 at 09:37:32 UTC from IEEE Xplore. Restrictions apply.

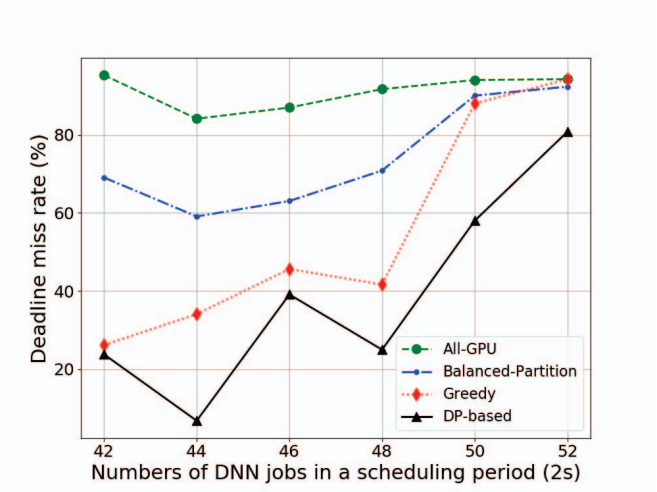


Fig. 11. The deadline miss rate under different numbers of tasks in a scheduling period (2s)

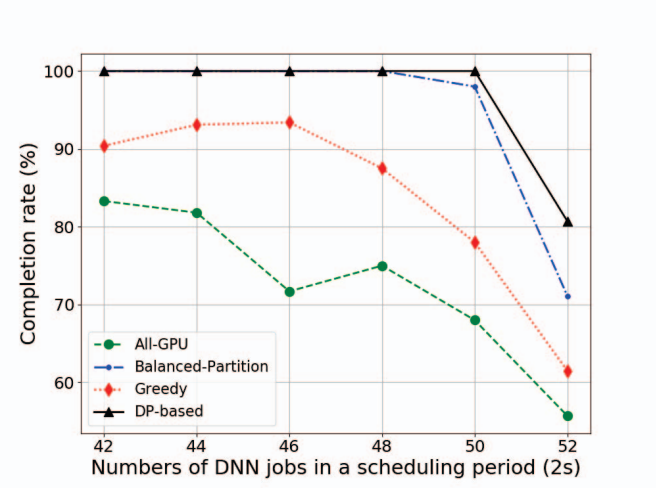


Fig. 12. The completion rate under different numbers of tasks in a scheduling period (2s)

as the 20-DNN-job set in our case (deadline miss rates are high). In the second set of experiments, we set the scheduling period as 2 second. The number of DNN jobs is from 42 to 52 in a scheduling period. Results (Fig. 11 and 12) show that the DP-based algorithm outperforms baselines in terms of deadline miss rate and completion rate in the second set of experiments.

VII. Related work

AI inference at the edge. Recent work has focused on accelerating AI pipelines by executing AI inference jobs at the edge, where data are generated and captured. Meanwhile, industries have released many powerful AI chips and devices for edge nodes and servers [25]. For example, the newest Nvidia AGX Xavier comprises an integrated 512-core Nvidia Volta GPU, bringing powerful computing resources to the edge. Yang *et al*. [1] propose some techniques, such as the parallel-pipeline execution of CNN jobs, to improve GPU utilization in Nvidia Drive PX2. DART [26] employs a pipeline-based scheduling architecture to improve worst-case response time and overall throughput of DNN tasks in edge devices (Intel Xeon and Nvidia TX2). LaLaRAND [27] presents a CPU-friendly quantization and fine-grained CPU/GPU allocation scheme to provide timing guarantees while with limited sacrifice on inference accuracy (experiments in Nvidia Jetson Xavier platform). Mobile devices are also considered as one type of edge nodes, while AI chips (such as GPU) are not as powerful as edge nodes mentioned above. Only few papers investigate how AI inference performs and works on mobile platforms. Heimdall [8] proposes a flexible scheduling framework for concurrent multi-DNN and rendering tasks on mobile GPU.

Real-time AI inference. Real-time requirements constitute an important aspect in mobile and edge AI inference due to the time-critical nature of many applications and systems. Recent work focused on real-time edge computing in some emerging directions, such as self-driving vehicles and Artificial Intelligence of Things (AIoT). Edgent [28] introduces a device-edge collaboration scheme and an early-exit mechanism to achieve real-time edge intelligence. RTDeepIoT [29] presents a real-time scheduling algorithm for AI inference by casting DNN as the imprecise computation with a mandatory part and several optional parts. Liu *et al*. [30] propose an online real-time scheduling architecture in the autonomous driving scenario, which renders higher priorities to mission-critical tasks.

VIII. Conclusion

In this paper, we presented a novel real-time AI inference framework for mobile devices with heterogeneous processors. We show that the pipelined design across mobile GPUs and CPUs can utilize computation resources more effectively and efficiently. The proposed real-time scheduler achieved a maximum utility with a DP-based approach. We have implemented our framework with some extensions to TensorFlow Lite on mobile platforms. In the future, we intend to include more dedicated mobile AI processors, such as Neural Processing Units (NPU), into the real-time AI inference framework. The collaborative execution between multiple heterogeneous mobile processors could inspire more scheduling and optimization opportunities.

Reference

[1] Ming Yang, Shige Wang, Joshua Bakita, Thanh Vu, F Donelson Smith, James H Anderson, and Jan-Michael Frahm. Re-thinking cnn frameworks for time-sensitive autonomous-driving applications: Addressing an industrial challenge. In *2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 305–317. IEEE, 2019.

[2] Bo-hu Li, Bao-cun Hou, Wen-tao Yu, Xiao-bing Lu, and Chun-wei Yang. Applications of artificial intelligence in intelligent manufacturing: a review. *Frontiers of Information Technology & Electronic Engineering*, 18(1):86–96, 2017.

[3] Nicholas D Lane and Petko Georgiev. Can deep learning revolutionize mobile sensing? In *Proceedings of the 16th international workshop on mobile computing systems and applications*, pages 117–122, 2015.

Authorized licensed use limited to: SHERIDAN LIBRARY. Downloaded on September 17,2026 at 09:37:32 UTC from IEEE Xplore. Restrictions apply.

[4] Shuochao Yao, Shaohan Hu, Yiran Zhao, Aston Zhang, and Tarek Abdelzaher. Deepsense: A unified deep learning framework for time-series mobile sensing data processing. In *Proceedings of the 26th International Conference on World Wide Web*, pages 351–360, 2017.

[5] Shuochao Yao, Yiran Zhao, Huajie Shao, Aston Zhang, Chao Zhang, Shen Li, and Tarek Abdelzaher. Rdeepsense: Reliable deep mobile computing models with uncertainty estimations. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 1(4):1–26, 2018.

[6] Hongzi Zhu, Yanyong Zhang, Mo Li, Ashwin Ashok, and Kaoru Ota. Exploring deep learning for efficient and reliable mobile sensing. *IEEE Network*, 32(4):6–7, 2018.

[7] Ke Zhang, Supeng Leng, Yejun He, Sabita Maharjan, and Yan Zhang. Mobile edge computing and networking for green and low-latency internet of things. *IEEE Communications Magazine*, 56(5):39–45, 2018.

[8] Juheon Yi and Youngki Lee. Heimdall: mobile gpu coordination platform for augmented reality applications. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*, pages 1–14, 2020.

[9] Andrey Ignatov, Radu Timofte, William Chou, Ke Wang, Max Wu, Tim Hartley, and Luc Van Gool. Ai benchmark: Running deep neural networks on android smartphones. In *Proceedings of the European Conference on Computer Vision (ECCV) Workshops*, pages 0–0, 2018.

[10] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.

[11] Thaha Mohammed, Carlee Joe-Wong, Rohit Babbar, and Mario Di Francesco. Distributed inference acceleration with adaptive dnn partitioning and offloading. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*, pages 854–863. IEEE, 2020.

[12] Zongqing Lu, Swati Rallapalli, Kevin Chan, and Thomas La Porta. Modeling the resource requirements of convolutional neural networks on mobile devices. In *Proceedings of the 25th ACM international conference on Multimedia*, pages 1663–1671, 2017.

[13] Eduardo Cuervo, Alec Wolman, Landon P Cox, Kiron Lebeck, Ali Razeen, Stefan Saroiu, and Madanlal Musuvathi. Kahawai: High-quality mobile gaming using gpu offload. In *Proceedings of the 13th Annual International Conference on Mobile Systems, Applications, and Services*, pages 121–135, 2015.

[14] Husheng Zhou, Soroush Bateni, and Cong Liu. Sˆ 3dnn: Supervised streaming and scheduling for gpu-accelerated real-time dnn workloads. In *2018 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 190–201. IEEE, 2018.

[15] Peifeng Yu and Mosharaf Chowdhury. Fine-grained gpu sharing primitives for deep learning applications. *Proceedings of Machine Learning and Systems*, 2:98–111, 2020.

[16] Aditya Dhakal, Sameer G Kulkarni, and KK Ramakrishnan. Machine learning at the edge: efficient utilization of limited cpu/gpu resources by multiplexing. In *2020 IEEE 28th International Conference on Network Protocols (ICNP)*, pages 1–6. IEEE, 2020.

[17] Siqi Wang, Gayathri Ananthanarayanan, Yifan Zeng, Neeraj Goel, Anuj Pathania, and Tulika Mitra. High-throughput cnn inference on embedded arm big. little multicore processors. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 39(10):2254–2267, 2019.

[18] Richard E Bellman and Stuart E Dreyfus. *Applied dynamic programming*, volume 2050. Princeton university press, 2015.

[19] Florian Schroff, Dmitry Kalenichenko, and James Philbin. Facenet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 815–823, 2015.

[20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[21] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.

[22] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016.

[23] Alex Kendall, Matthew Grimes, and Roberto Cipolla. Posenet: A convolutional network for real-time 6-dof camera relocalization. In *Proceedings of the IEEE international conference on computer vision*, pages 2938–2946, 2015.

[24] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L Yuille. Semantic image segmentation with deep convolutional nets and fully connected crfs. *arXiv preprint arXiv:1412.7062*, 2014.

[25] Stephan Patrick Baller, Anshul Jindal, Mohak Chadha, and Michael Gerndt. Deepedgebench: Benchmarking deep neural networks on edge devices. In *2021 IEEE International Conference on Cloud Engineering (IC2E)*, pages 20–30. IEEE, 2021.

[26] Yecheng Xiang and Hyoseung Kim. Pipelined data-parallel cpu/gpu scheduling for multi-dnn real-time inference. In *2019 IEEE Real-Time Systems Symposium (RTSS)*, pages 392–405. IEEE, 2019.

[27] Woosung Kang, Kilho Lee, Jinkyu Lee, Insik Shin, and Hoon Sung Chwa. Lalarand: Flexible layer-by-layer cpu/gpu scheduling for real-time dnn tasks. In *2021 IEEE Real-Time Systems Symposium (RTSS)*, pages 329–341. IEEE, 2021.

[28] En Li, Liekang Zeng, Zhi Zhou, and Xu Chen. Edge ai: On-demand accelerating deep neural network inference via edge computing. *IEEE Transactions on Wireless Communications*, 19(1):447–457, 2019.

[29] Shuochao Yao, Yifan Hao, Yiran Zhao, Huajie Shao, Dongxin Liu, Shengzhong Liu, Tianshi Wang, Jinyang Li, and Tarek Abdelzaher. Scheduling real-time deep learning services as imprecise computations. In *2020 IEEE 26th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, pages 1–10. IEEE, 2020.

[30] Shengzhong Liu, Shuochao Yao, Xinzhe Fu, Rohan Tabish, Simon Yu, Ayoosh Bansal, Heechul Yun, Lui Sha, and Tarek Abdelzaher. On removing algorithmic priority inversion from mission-critical machine inference pipelines. In *2020 IEEE Real-Time Systems Symposium (RTSS)*, pages 319–332. IEEE, 2020.

Authorized licensed use limited to: SHERIDAN LIBRARY. Downloaded on September 17,2026 at 09:37:32 UTC from IEEE Xplore. Restrictions apply.